

Introduzione

Python non è solo un linguaggio facile per iniziare, ma offre anche potenti strumenti per automatizzare compiti, testare il codice, registrare eventi (logging), ottimizzare le prestazioni e gestire l'esecuzione concorrente. In questa lezione introduttiva esploreremo insieme come sfruttare queste capacità, assumendo che tu conosca già le basi di Python ma sia agli inizi su questi argomenti più avanzati. L'obiettivo è mostrarti in modo chiaro e pratico come **automatizzare operazioni di sistema**, creare **log** per monitorare i tuoi programmi, effettuare **debug** efficacemente, scrivere **test** automatici, profilare le **prestazioni**, utilizzare la programmazione **asincrona (AsyncIO)** e preparare il tuo codice per il **packaging e il deployment**.

Affronteremo ogni tema in modo progressivo: partiremo dallo scripting di automazione con moduli come `os`, `pathlib`, `shutil` e `subprocess`, per poi accennare alla pianificazione di script con scheduler o **cron**. Vedremo come utilizzare il modulo `logging` per registrare informazioni utili durante l'esecuzione di un programma e come effettuare il **debugging** usando stampe, asserzioni e il debugger interattivo `pdb`. Passeremo quindi al **testing** del codice con `unittest` (e un cenno a `pytest`), e a come misurare e migliorare le **performance** con strumenti come `time`, `timeit` e `cProfile`. Infine, introdurremo la programmazione **AsyncIO** (asincrona) con `async/await`, e come **"impacchettare"** un progetto Python (con `setup.py` o `pyproject.toml`) e distribuirlo con `pip`, PyPI o creare eseguibili standalone.

```
# Questa lezione mostra come automatizzare compiti, testare codice, fare
logging, ecc.
# (Il codice seguente è solo un messaggio di benvenuto di esempio)
print("Benvenuto! Preparati a scoprire come rendere Python ancora più
potente.")
```

Modulo os: variabili d'ambiente e directory corrente

Un primo passo per automatizzare compiti con Python è interagire con il sistema operativo. Il modulo integrato `os` fornisce funzioni per ottenere informazioni e gestire l'ambiente di esecuzione del programma. Ad esempio, con `os.getcwd()` possiamo sapere qual è la *current working directory* (directory di lavoro corrente) del nostro script, mentre `os.environ` permette di leggere e impostare le **variabili d'ambiente** del sistema. Le variabili d'ambiente sono valori (come percorsi o credenziali) definiti a livello di sistema, e Python può accedervi tramite questo dizionario speciale.

Possiamo usare `os.environ.get("NOME")` per ottenere il valore di una variabile (restituisce `None` se non esiste) oppure assegnare `os.environ["NOME"] = "valore"` per impostarne una nuova (valida solo per il processo corrente). Ecco un esempio che stampa la directory corrente e gestisce una variabile d'ambiente:

```
import os
print("Directory corrente:", os.getcwd()) # es. '/home/utente/progetto'
```

```
os.environ["SALUTO"] = "Ciao"
print("Variabile d'ambiente SALUTO:", os.environ["SALUTO"])
```

Modulo os: gestione di file e directory

Il modulo `os` include anche molte funzioni per **operare su file e directory**, utilissime nello scripting di automazione. Ad esempio, `os.listdir(path)` restituisce l'elenco dei file in una directory, `os.mkdir(path)` crea una nuova cartella, `os.rename(src, dst)` rinomina un file o cartella, mentre `os.remove(file)` e `os.rmdir(dir)` permettono di eliminare rispettivamente file (o directory vuote). Queste funzioni permettono a uno script Python di creare strutture di cartelle, spostare file o ripulire file indesiderati senza intervento manuale.

È importante usare queste funzioni con cautela (soprattutto quelle di rimozione) e magari aggiungere controlli (come verificare se un file esiste con `os.path.exists`) prima di compiere azioni distruttive. Nel seguente esempio creiamo una directory, ne elenchiamo il contenuto, la rinominiamo e infine la cancelliamo, mostrando alcune operazioni di base:

```
import os

os.mkdir("nuova_cartella")          # crea una nuova directory
print("Contenuto corrente:", os.listdir("."))
# lista i file nella directory attuale
os.rename("nuova_cartella", "cartella_rinominata") # rinomina la directory
os.rmdir("cartella_rinominata")      # rimuove la directory (deve
essere vuota)
print("Contenuto finale:", os.listdir("."))
```

Modulo os: percorrere directory con os.walk

Spesso, durante l'automazione, serve **visitare ricorsivamente** tutti i file in una struttura di cartelle (ad esempio per cercare file con un certo nome o per applicare un'operazione su ogni file trovato). Il modulo `os` offre la funzione `os.walk()`, che genera, passo dopo passo, le informazioni di ogni directory all'interno di un albero di cartelle. Ad ogni iterazione, `os.walk` restituisce una tupla (percorso_corrente, sottocartelle, file) relativa a una directory diversa.

Possiamo usare `os.walk` in un loop `for` per attraversare automaticamente tutte le sottodirectory di un percorso di partenza. In questo modo uno script può "camminare" dentro le directory senza doverle specificare manualmente. Nell'esempio seguente usiamo `os.walk` per stampare i file presenti in ogni cartella, partendo dalla directory corrente `"."`:

```
import os

for percorso, cartelle, files in os.walk("."):
    print(f"Nella cartella {percorso} ci sono file: {files}")
```

(Nel codice sopra, `cartelle` è la lista delle sottocartelle trovate in `percorso`, e `files` la lista dei file. Possiamo usare queste informazioni, ad esempio, per applicare operazioni specifiche a determinati file.)

Automazione: ricerca di file con glob

Un'altra esigenza comune negli script di automazione è **trovare file che corrispondono a un certo pattern**, ad esempio tutti i file con estensione `.txt` o con un prefisso particolare. Python fornisce il modulo `glob` per effettuare ricerche di file utilizzando caratteri jolly (wildcard) come `*` e `?` analogamente a come si fa nel terminale.

La funzione `glob.glob(pattern)` restituisce la lista dei percorsi che corrispondono al pattern specificato. Ad esempio, `"*.txt"` corrisponde a "tutti i file che terminano con `.txt` nella directory corrente". Questo strumento è molto utile per selezionare insiemi di file su cui operare senza dover specificare ognuno manualmente. Ecco un semplice esempio che trova e stampa tutti i file di testo nella directory corrente:

```
import glob

files_testo = glob.glob("*.txt")
print("File di testo trovati:", files_testo)
```

(Se nessun file corrisponde al pattern, `glob.glob` restituisce una lista vuota `[]`. Inoltre, possiamo usare percorsi più complessi, ad esempio `"../data/*.csv"` per trovare tutti i CSV in una cartella sorella.)

Pathlib: gestione dei percorsi in modo avanzato

Python 3 include il modulo `pathlib`, che introduce l'oggetto `Path` per rappresentare e manipolare i **percorsi di file e directory** in modo orientato agli oggetti. `pathlib` rende il codice più leggibile e portabile rispetto all'uso di `os.path`, perché possiamo combinare parti di percorso usando operatori (`/`) e chiamare metodi su un oggetto `Path` per varie operazioni.

Ad esempio, con `Path("cartella") / "file.txt"` otteniamo un percorso figlio (simile a `os.path.join`). Possiamo poi verificare se un file esiste con `path.exists()`, controllare se è un file o una directory con `path.is_file()` / `path.is_dir()`, leggere o scrivere file direttamente con metodi come `path.read_text()` o `path.write_text()`. Nell'esempio seguente costruiamo un percorso e verifichiamo alcune sue proprietà:

```
from pathlib import Path

p = Path("cartella") / "sottocartella" / "file.txt"
# costruisce un percorso annidato
print("Percorso creato:", p)
print("Il file esiste?", p.exists())
print("Nome del file:", p.name, "- Estensione:", p.suffix)
```

(Nell'esempio, `p.name` darebbe `"file.txt"` e `p.suffix` `".txt"`. Se il percorso `cartella/sottocartella` non esiste, `p.exists()` restituirà `False`. Possiamo creare le cartelle necessarie con `Path.mkdir(parents=True)` se serve.)

Modulo shutil: operazioni di copia e spostamento file

Il modulo `shutil` offre funzioni di alto livello per operare sui file, complementari a quelle di `os`. In particolare, `shutil.copy(src, dst)` copia un file da `src` a `dst` (mantenendone il contenuto), mentre `shutil.move(src, dst)` permette di spostare/rinominare file o intere directory in una singola operazione. `shutil.rmtree(dir)` può rimuovere ricorsivamente una directory con tutto il suo contenuto (**attenzione**: è distruttiva come un `rm -rf`).

Queste funzioni semplificano compiti di backup, duplicazione di file, organizzazione di cartelle, rispetto a dover leggere e scrivere file manualmente. Nell'esempio seguente copiamo un file e poi lo spostiamo in una sottocartella (assumendo che quest'ultima esista già):

```
import shutil

shutil.copy("documento.txt", "copia_documento.txt") # copia il file
documento.txt
print("File copiato come copia_documento.txt")
shutil.move("copia_documento.txt", "archivio/copia_documento.txt") # sposta
il file nella cartella 'archivio'
print("File spostato nella cartella 'archivio'")
```

(Se la cartella di destinazione non esiste, `shutil.move` solleverà un errore. Possiamo creare la cartella prima con `os.makedirs("archivio", exist_ok=True)`. Inoltre, `shutil.copytree(src, dst)` permette di copiare intere directory in blocco.)

Modulo subprocess: eseguire comandi esterni

Spesso uno script Python deve interagire con altri programmi o eseguire comandi di sistema. Il modulo `subprocess` consente di lanciare processi esterni dal nostro script e, volendo, catturarne l'output. Ad esempio, possiamo usare `subprocess.run()` per eseguire un comando come faremmo nel terminale. Questo è utile per automatizzare sequenze di comandi shell o integrare strumenti esterni nel workflow Python.

Con `subprocess.run` possiamo passare il comando come lista di argomenti (`["comando", "arg1", ...]`) o come stringa (impostando `shell=True` per farlo eseguire dalla shell di sistema). Nel prossimo snippet eseguiamo un semplice comando **echo** per stampare un messaggio (su Windows utilizziamo `shell=True` affinché funzioni il built-in della shell):

```
import subprocess

# Esegue un comando di sistema (qui 'echo' stampa un messaggio tramite la
shell)
subprocess.run("echo Ciao dal subprocess!", shell=True)
```

(In ambiente Unix, potremmo usare `subprocess.run(["echo", "Hello"])` senza shell. Possiamo anche catturare l'output aggiungendo `capture_output=True, text=True` e leggere

`result.stdout`. Ad esempio, `result = subprocess.run(["ls"], capture_output=True, text=True)` ci permetterebbe di ottenere l'elenco dei file in `result.stdout`.)

Esempio pratico: script di automazione file

Dopo aver visto singoli strumenti, ecco un esempio concreto di **script di automazione**. Supponiamo di voler fare il backup di tutti i file di testo da una cartella `documenti` in una cartella `backup`. Lo script deve creare la cartella di destinazione se non esiste, copiare tutti i file con estensione `.txt` e magari stampare (o registrare) ogni copia effettuata. Possiamo combinare `pathlib` per gestire i percorsi e trovare i file, con `shutil.copy` per copiarli.

Nel codice seguente usiamo `Path.glob("*.txt")` per ottenere tutti i file `.txt` nella cartella origine, creiamo la cartella di backup con `mkdir(exist_ok=True)` e copiamo ogni file mantenendo lo stesso nome. Ogni operazione viene confermata con una stampa su console:

```
from pathlib import Path
import shutil

origine = Path("documenti")
destinazione = Path("backup")
destinazione.mkdir(exist_ok=True) # crea la cartella di backup se non esiste

for file_txt in origine.glob("*.txt"):
    shutil.copy(file_txt, destinazione / file_txt.name)
    print(f"Copiato {file_txt.name} in {destinazione}")
```

(Questo script effettuerà il backup di tutti i `.txt` presenti in `documenti`. In uno scenario reale, potremmo migliorarlo aggiungendo gestione delle eccezioni e magari registrando l'esito su un file di log invece di usare `print`.)

Scripting: rendere uno script eseguibile (shebang)

Quando scriviamo uno script Python per automatizzare un'attività, potremmo volerlo eseguire direttamente come un programma dal sistema (almeno su sistemi Unix/Linux). Per fare ciò, all'inizio del file possiamo aggiungere una **shebang line**, ad esempio `#!/usr/bin/env python3`, che indica al sistema di usare l'interprete Python. Inoltre, dobbiamo dare permessi di esecuzione al file (`chmod +x nome_script.py`). In questo modo, potremo avviare lo script semplicemente con `./nome_script.py`.

Un'altra buona pratica è includere `if __name__ == "__main__":` nel nostro script. Questo controllo fa sì che il codice in esso contenuto venga eseguito solo se il file è eseguito direttamente, e non quando è importato come modulo in altri programmi. Ecco come potrebbe apparire l'inizio di uno script Python eseguibile:

```
#!/usr/bin/env python3
# Shebang sopra permette di eseguire lo script direttamente su Unix

def main():
```

```
print("Esempio di script Python eseguibile direttamente")

if __name__ == "__main__":
    main() # esegue la funzione main solo se lanciato come script
```

(Su Windows la shebang viene ignorata, ma se l'estensione `.py` è associata a Python, si può eseguire lo script con doppio click o usando il launcher `py`. Il blocco `__main__` è utile in ogni caso per evitare esecuzioni indesiderate quando si importa lo script in altri moduli.)

Scheduling: esecuzione periodica con ciclo e sleep

Oltre ad eseguire script su richiesta, spesso c'è bisogno di **automatizzare l'esecuzione periodica** di un'attività (ad esempio un backup giornaliero). Un modo semplice in Python è scrivere uno script che rimanga in esecuzione continua e utilizzi `time.sleep()` per attendere intervalli di tempo tra un'esecuzione e l'altra. Ad esempio, un loop infinito che esegue un certo task e poi dorme per tot secondi.

Questo approccio non richiede nulla di esterno: il programma stesso gestisce il timing. Tuttavia, bisogna ricordare che, così facendo, il processo Python rimane attivo continuamente. Nell'esempio seguente, lo script stampa un messaggio (rappresentativo di un task) ogni 5 secondi. Interromperlo manualmente (Ctrl+C) fermerà il loop:

```
import time

while True:
    print("Esecuzione task periodico...")
    # ... qui il codice del task da eseguire ...
    time.sleep(5) # attende 5 secondi prima della prossima iterazione
```

(Questo schema è utile per semplici scheduler interni all'applicazione. Per compiti più complessi o robusti, si può valutare l'uso di librerie di scheduling o meccanismi di sistema come cron, discussi a seguire.)

Scheduling: libreria schedule per pianificare task

Invece di gestire manualmente i tempi con `sleep`, possiamo usare una libreria Python come **schedule** (non facente parte della libreria standard) che offre un'interfaccia semplice per pianificare l'esecuzione di funzioni a intervalli regolari. La libreria `schedule` permette di definire job tipo "esegui questa funzione ogni ora" o "ogni giorno alle 12:00". È molto utile per piccoli script multiplatforma (funziona anche su Windows, dove non c'è cron).

Per usarla, prima installiamola (`pip install schedule`). Poi definiamo il task come funzione e utilizziamo metodi come `schedule.every(1).hour.do(task)` per schedularlo. Infine, serve un loop che periodicamente richiama `schedule.run_pending()` per eseguire i task quando è il momento. Ecco un esempio che pianifica un semplice saluto ogni ora:

```
# Prima installare con: pip install schedule
import schedule
```

```
import time

def saluta():
    print("Ciao! Task schedulato eseguito.")

schedule.every(1).hour.do(saluta)    # pianifica la funzione ogni ora

while True:
    schedule.run_pending()    # esegue i task schedulati se è l'ora
    time.sleep(1)            # attende un secondo prima di ricontrollare
```

(La libreria `schedule` semplifica molto la sintassi rispetto a `cron`. Tieni presente che lo script deve rimanere attivo per eseguire i job: se il programma termina, i task non verranno eseguiti fino al successivo avvio.)

Scheduling: cenni sui Cron job

Su sistemi Unix/Linux il metodo più comune per eseguire script a intervalli regolari è usare **cron**, un demone di sistema che lancia comandi secondo una pianificazione definita (i "cron job"). Possiamo far eseguire uno script Python ogni giorno o ogni ora configurando opportunamente `cron`, senza dover tenere il nostro script in esecuzione continua. Per aggiungere un cron job, si utilizza il comando `crontab -e` e si inserisce una linea con la sintassi del cron (minuto, ora, giorno del mese, mese, giorno della settimana, comando).

Ad esempio, per eseguire uno script Python ogni giorno alle 8:00, possiamo aggiungere al `crontab`:

```
# Esempio di riga nel crontab (Linux/macOS):
0 8 * * * /usr/bin/python3 /home/utente/backup.py
# (esegue backup.py ogni giorno alle 8:00)
```

(I primi cinque campi `0 8 * * *` specificano il tempo: minuto 0, ora 8, tutti i giorni. Assicuriamoci di mettere il percorso completo dell'interprete Python e dello script. Su Windows, `cron` non è disponibile nativamente, ma c'è l'Utilità di Pianificazione attività che offre funzionalità simili tramite interfaccia grafica o comando `schtasks`.)

Logging: perché utilizzare un sistema di log

Durante lo sviluppo e l'esecuzione di programmi, è fondamentale **tracciare cosa accade**: per questo si usano i log. Molti principianti iniziano inserendo `print()` per capire il flusso, ma un sistema di logging è più flessibile e potente. Il logging consente di mantenere uno storico delle operazioni, classificare i messaggi per severità (debug, info, warning, error, ...) e decidere facilmente dove inviare questi messaggi (console, file, ecc.), il tutto senza modificare il codice principale dell'applicazione una volta configurato.

In Python, il modulo integrato `logging` fornisce un sistema unificato per generare e gestire log. Usare `logging` al posto di semplici `print` consente di **disabilitare o filtrare** i messaggi a seconda del livello di importanza, senza dover rimuovere manualmente istruzioni di stampa. Inoltre, in ambienti di produzione è comune scrivere i log su file, per poterli consultare dopo l'esecuzione del programma o in caso di errori. Ecco un semplice esempio di utilizzo del logging per emettere un messaggio di avviso:

```
import logging
logging.warning("Questo è un messaggio di warning di esempio.")
```

(Per default, il logging mostra in console solo messaggi di livello **WARNING** o superiore. Il messaggio sopra comparirà preceduto dal livello e altre info. Vediamo nei prossimi slide come configurare i log in modo personalizzato.)

Logging: utilizzo base del modulo logging

L'utilizzo di base del logging in Python tipicamente consiste nel chiamare `logging.basicConfig()` all'inizio del programma per configurare il comportamento di logging (livello minimo di messaggi, formato, e destinazione come stdout o file). Ad esempio, possiamo impostare il livello di log a `INFO` per vedere sia messaggi di info che warning/error. Dopo la configurazione, usiamo funzioni come `logging.debug()`, `logging.info()`, `logging.warning()`, `logging.error()` e `logging.critical()` per registrare messaggi ai vari livelli.

Per esempio, configuriamo il logging a livello `INFO` (il default sarebbe `WARNING`) e proviamo a inviare un messaggio informativo e uno di errore. Il messaggio di info ora sarà visibile, mentre senza configurazione non lo sarebbe stato. Questo permette di controllare la verbosità dell'output senza cambiare tutte le chiamate di log nel codice:

```
import logging

logging.basicConfig(level=logging.INFO) # configurazione base: livello INFO
logging.info("Inizio del programma")   # messaggio di livello INFO
logging.error("Si è verificato un errore.") # messaggio di livello ERROR
```

(Per default, `basicConfig` senza parametri logga su console. Possiamo specificare parametri come `filename` per log su file, `filemode` per modo di apertura file, e `format` per personalizzare il formato del messaggio. Vediamolo a seguire.)

Logging: livelli di log e configurazione

Il modulo logging definisce vari **livelli di gravità** dei messaggi: i principali in ordine crescente sono **DEBUG**, **INFO**, **WARNING**, **ERROR** e **CRITICAL**. Quando configuriamo il logging, possiamo impostare un livello soglia: verranno gestiti solo i messaggi di quel livello o superiori (livelli più gravi). Ad esempio, se il livello è `INFO`, i debug vengono ignorati, se è `WARNING` verranno mostrati solo warning, errori e critical, e così via. Il livello di default (se non chiamiamo `basicConfig`) è `WARNING`.

Possiamo immaginare i livelli come priorità: **DEBUG** per dettagli di diagnostica utili agli sviluppatori, **INFO** per informazioni sul normale funzionamento, **WARNING** per situazioni anomale ma non critiche, **ERROR** per errori runtime che permettono comunque al programma di continuare, e **CRITICAL** per errori gravi dopo i quali l'app potrebbe non poter proseguire correttamente. Ecco un esempio di emissione di messaggi a tutti i livelli (impostiamo il livello a `DEBUG` per vederli tutti):

```
import logging
```



```
logging.basicConfig(level=logging.DEBUG)
logging.debug("Messaggio di debug (dettagli di diagnostica)")
logging.info("Messaggio informativo di funzionamento")
logging.warning("Messaggio di warning: situazione inattesa")
logging.error("Messaggio di errore")
logging.critical("Messaggio di errore critico!")
```

(In un'applicazione reale si sceglie il livello di log in base all'ambiente: ad es. DEBUG o INFO in sviluppo per avere molti dettagli, WARNING o ERROR in produzione per evitare troppi messaggi. I log debug possono includere variabili interne e flussi di esecuzione utili solo allo sviluppatore.)

Logging: salvare i log su file

Oltre (o in alternativa) a mostrare i log in console, è spesso utile **scriverli su un file** per conservarli. Questo è particolarmente importante per applicazioni che girano in background o su server, dove non c'è sempre un terminale visibile. Con `logging.basicConfig` possiamo specificare il parametro `filename="nomefile.log"` per redirigere automaticamente tutti i log su quel file. Possiamo anche usare diversi **handler** per inviare alcuni log su file e altri su console, ma come base il file unico va bene.

Nell'esempio seguente configuriamo il logging per scrivere su un file `app.log` tutti i messaggi di livello INFO e superiori. Dopodiché logghiamo un messaggio info e uno di errore: non appariranno in console, ma verranno aggiunti al file `app.log`. Aprendo il file, vedremo le linee corrispondenti ai nostri messaggi (tipicamente precedute dal livello e magari timestamp di default):

```
import logging

logging.basicConfig(filename='app.log', level=logging.INFO)
logging.info("Questo messaggio verrà scritto nel file app.log")
logging.error("Anche questo messaggio andrà nel file di log.")
```

(Quando si usano file di log, attenzione: per default `basicConfig` apre il file in append. Se vogliamo sovrascriverlo a ogni esecuzione possiamo passare `filemode='w'`. Inoltre, il modulo logging permette di avere rotazione dei file (log rotation) con handler appositi, per evitare che il file cresca indefinitamente.)

Logging: formattare i messaggi di log

Il sistema di logging permette anche di **personalizzare il formato** dei messaggi, aggiungendo ad esempio timestamp, nome del modulo o del file che ha generato il log, ecc. Questo si fa passando il parametro `format` a `logging.basicConfig`. Nel formato possiamo usare delle variabili speciali: ad esempio `%(asctime)s` per la data/ora, `%(levelname)s` per il nome del livello (INFO, ERROR, ecc.), `%(message)s` per il testo del messaggio loggato, e molte altre (come il nome del file, linea di codice, thread, ecc.).

Nel seguente esempio configuriamo un formato che include timestamp, livello e messaggio. Impostiamo il livello a WARNING giusto per prova, e generiamo un warning. L'output nel log (in console in questo caso) conterrà la data e ora corrente, il livello WARNING e il messaggio:

```
import logging

logging.basicConfig(level=logging.WARNING,
                    format="%(asctime)s - %(levelname)s - %(message)s")
logging.warning("Messaggio con timestamp e livello nel formato specificato")
```

(Il timestamp di default `%(asctime)s` è in formato `YYYY-MM-DD HH:MM:SS,milliseconds`. Possiamo personalizzarlo ulteriormente con `datefmt` in `basicConfig` se necessario. Con questi strumenti i log diventano molto più leggibili e utili per analisi successive.)

Logging: registrare eccezioni e stack trace

Un caso particolare di logging è la registrazione delle **eccezioni non gestite** o degli errori con il relativo **traceback** (traccia dello stack). Anziché stampare a console la stack trace, possiamo usare `logging.exception(msg)` all'interno di un blocco `except`. Questa funzione loggherà il messaggio specificato *insieme* alla traccia completa dell'eccezione corrente (equivalente a ciò che verrebbe stampato da `traceback.print_exc()`). Il livello usato da `logging.exception` è, per default, `ERROR`.

Usare `logging.exception` dentro un `except` è un modo comodo per tenere traccia di errori avvenuti in produzione: si avrà nel file di log sia l'indicazione dell'errore sia la linea di codice dove è avvenuto, ecc. Nell'esempio seguente forziamo un errore di divisione per zero e lo logghiamo su file:

```
import logging

logging.basicConfig(filename='error.log', level=logging.ERROR)
try:
    1 / 0
except ZeroDivisionError:
    logging.exception("Divisione per zero rilevata!")
```

(Dopo aver eseguito questo codice, il file `error.log` conterrà il messaggio "Divisione per zero rilevata!" seguito dal traceback completo dell'eccezione `ZeroDivisionError`. In generale, è buona pratica loggare le eccezioni inattese in modo da poterle analizzare a posteriori.)

Debugging: cos'è il debugging e perché è importante

Debugging significa letteralmente "cacciare i bug": è il processo di identificare e risolvere problemi o errori (bug) nel codice. Anche con test e pianificazione, ogni programmatore dedica molto tempo al debugging. Capire come farlo efficacemente è cruciale. Il debugging include metodi semplici come inserire stampe o asserzioni per verificare lo stato del programma in punti chiave, fino all'uso di strumenti dedicati (debugger interattivi) che permettono di eseguire il codice passo-passo e ispezionare le variabili.

Il motivo per cui il debugging è importante è che raramente il codice funziona correttamente al primo tentativo per casi complessi. Sapere interpretare un **traceback** (la traccia d'errore mostrata quando il programma lancia un'eccezione non gestita) o usare un debugger per fermare l'esecuzione in un punto e analizzare la situazione ci aiuta a capire la causa di un malfunzionamento. Qui di seguito vedremo

alcuni approcci: l'uso di `print` e `assert` per rapidi controlli, la lettura delle tracce di errore, e l'utilizzo del modulo `pdb` (Python Debugger) per un debug interattivo.

```
# Esempio: funzione con un bug
def somma(a, b):
    return a - b # BUG: dovrebbe restituire a + b, ma sottrae

ris = somma(5, 3)
print("Risultato ottenuto:", ris) # Risultato errato: 2 (ci aspettiamo 8)
```

(Nell'esempio sopra, abbiamo introdotto intenzionalmente un bug. Il risultato stampato non è quello atteso. Questo simula una tipica situazione da debug: il programma gira ma dà un output sbagliato, quindi dobbiamo indagare perché.)

Debugging: stampa di debug e assert

Il metodo più immediato per fare debugging è inserire delle **stampe (print)** nel codice per verificare il valore delle variabili o il flusso di esecuzione. Ad esempio, se una variabile ha un valore inaspettato, possiamo stamparla in vari punti per tracciare come cambia. Questo metodo "artigianale" è semplice ma efficace in molti casi. Un altro strumento utile è l'**assert**: un'asserzione è un controllo che ci aspettiamo sia vero in quel punto del programma; se la condizione è falsa, Python solleva un `AssertionError` fermando il programma e indicando la riga del fallimento. Gli assert sono ottimi per esprimere condizioni che dovrebbero essere garantite: se non lo sono, c'è un bug.

Nel codice seguente, usiamo `print` per stampare una variabile e due `assert` per verificare delle condizioni. Il primo assert passa ($10 > 0$ è True), il secondo fallisce ($x < 5$ è False se x è 10), causando un errore con il messaggio specificato. Gli assert sono di aiuto durante lo sviluppo e debug, ma in produzione possono essere disattivati avviando Python con l'opzione `-O`. Ecco un esempio:

```
x = 10
print(f"Debug: x = {x}")
assert x > 0, "x dovrebbe essere positivo"
print("Questo messaggio appare solo se l'assert precedente non fallisce")
assert x < 5, "x dovrebbe essere minore di 5" # Questo assert fallirà se x=10
print("Questa linea non verrà eseguita se l'assert fallisce")
```

(Quando l'assert su $x < 5$ fallisce, il programma si interrompe con un `AssertionError`. Il messaggio "x dovrebbe essere minore di 5" aiuta a identificare il problema. In un contesto di debugging, questo ci dice che x ha un valore inatteso in quel punto.)

Debugging: leggere le tracce di errore (traceback)

Quando un programma Python lancia un'eccezione non gestita, l'interprete stampa la **traceback**, ovvero la "traccia" dell'errore: essa mostra la sequenza di chiamate di funzione che hanno portato all'errore, indicando file, linee di codice e tipo di eccezione. Imparare a leggere il traceback è fondamentale: di solito l'ultima riga indica il tipo di errore e un messaggio (ad esempio `ZeroDivisionError: division by zero`), mentre le righe sopra risalgono alle chiamate

precedenti (il punto del codice dove quell'errore si è originato e la catena di funzioni che era in esecuzione).

In modalità interattiva o eseguendo uno script normalmente, la traceback viene stampata automaticamente quando c'è un errore. Se vogliamo catturarla e gestirla, possiamo usare il modulo `traceback`. Un pattern comune è usare `traceback.print_exc()` dentro un `except` per stampare la traccia senza fermare del tutto il programma. Nell'esempio sotto forziamo un errore (divisione per zero) dentro un try/except e usiamo `traceback` per stampare le informazioni sull'errore:

```
import traceback

def dividi(a, b):
    return a / b

try:
    risultato = dividi(5, 0)
except Exception as e:
    print("Si è verificato un errore:")
    traceback.print_exc()
```

(Il codice sopra intercetta qualsiasi eccezione generica `Exception` e stampa la relativa traceback. In questo caso vedremo l'errore `ZeroDivisionError` e la linea esatta dentro `dividi` che ha causato il problema. Questo approccio è utile per loggare gli errori senza interrompere il programma, magari all'interno di un server che deve continuare a girare.)

Debugging: uso del debugger pdb

Per un controllo più fine dell'esecuzione, Python fornisce un **debugger interattivo** chiamato `pdb` (Python DeBugger). Con `pdb` possiamo eseguire il programma un passo alla volta, impostare **breakpoint** (punti di interruzione) dove l'esecuzione si ferma dandoci controllo, ed esaminare/modificare variabili in tempo reale. Questo aiuta a capire cosa succede riga per riga. Possiamo avviare il debugger in diversi modi: uno è inserire `pdb.set_trace()` nel codice nel punto desiderato. In questo modo, quando il programma raggiunge quella linea, entrerà in modalità interattiva (nel terminale) permettendoci di inserire comandi `pdb` (come `n` per next step, `s` per step in funzione, `p` `variable` per stampare il valore di una variabile, etc.).

Ecco un esempio in cui usiamo `pdb.set_trace()`. Poniamo un breakpoint prima di una chiamata a funzione, così potremo, durante l'esecuzione, ispezionare i valori di `x` e `y`, e poi usare comandi `pdb` per avanzare ed esaminare `risultato`:

```
import pdb

def calcola(a, b):
    return a + b

x = 7
y = 3
```

```
pdb.set_trace() # Punto di interruzione: da qui possiamo esaminare x, y,
ecc.
risultato = calcola(x, y) # Eseguiamo passo-passo con il debugger
print("Risultato:", risultato)
```

(Quando eseguiamo questo script, entrerà nel prompt di pdb al momento di `set_trace()`. Da lì possiamo usare ad esempio `p x` (print x) per vedere `x=7`, `p y` per `y=3`, poi `n` per eseguire la riga successiva (la chiamata a `calcola`), e così via. Questo è molto potente per capire il flusso in punti critici. Per uscire dal debugger, comando `c` (continue) fino alla fine o `q` (quit).)

Debugging: breakpoint() e debugger negli IDE

Un'alternativa a `pdb.set_trace()` è l'uso di `breakpoint()`, funzione integrata a partire da Python 3.7. Chiamando `breakpoint()` il programma entrerà nel debugger predefinito (di default proprio pdb) in quel punto, senza dover importare pdb o chiamare `set_trace` manualmente. È un piccolo shortcut configurabile (tramite variabile d'ambiente si potrebbe usare un debugger diverso). Quindi possiamo usare semplicemente `breakpoint()` dove vogliamo fermare l'esecuzione per debug.

Va anche notato che molti **IDE (Integrated Development Environment)** come PyCharm, Visual Studio Code, Spyder, ecc., hanno un debugger grafico integrato. In un IDE possiamo impostare breakpoints cliccando a lato del codice, avviare il debug e usare interfacce visuali per controllare variabili, stack di chiamate, ecc., senza inserire comandi nel codice. Questo approccio è spesso più comodo per lunghe sessioni di debug. Ad esempio, in VSCode possiamo avviare il debug e vedere le variabili all'interruzione.

Nel frammento sotto usiamo `breakpoint()` all'interno di una funzione, per dimostrazione, che farà entrare nel debugger interattivo quando chiamata. Se eseguito in un IDE, potremmo invece gestire il tutto con i tool grafici dell'IDE stesso:

```
def moltiplica(a, b):
    prodotto = a * b
    breakpoint() # pausa l'esecuzione ed entra nel debugger interattivo
    return prodotto

moltiplica(4, 5)
```

(Nei moderni workflow di sviluppo, spesso si preferisce il debugger dell'IDE per comodità, ma è utile conoscere `pdb` e `breakpoint()` per quando ci si trova a debuggare in ambienti senza IDE, ad esempio direttamente su un server tramite SSH.)

Testing: importanza dei test automatici

Testare il codice significa verificare che le funzioni e le parti del programma si comportino come previsto. Farlo in modo automatico (tramite script di test) è fondamentale per mantenere la qualità del software man mano che cresce: i test automatici (unit test, integration test, ecc.) possono essere eseguiti frequentemente per assicurarsi che nuove modifiche non abbiano introdotto regressioni. Questo dà confidenza nello refactoring e agilità nello sviluppo.

Per un principiante con basi di Python, i test unitari sono un ottimo punto di partenza: un **unit test** verifica una singola unità di codice (di solito una funzione o metodo) in isolamento, controllando che per dati input otteniamo gli output attesi. Python include nel suo core library il modulo `unittest` che fornisce un framework per scrivere e aggregare test. Ci sono anche strumenti esterni come `pytest` che semplificano la sintassi dei test.

Un primo esempio di test potrebbe anche essere manuale: ad esempio, possiamo scrivere codice che chiama una funzione e controlla se il risultato è quello atteso, stampando un messaggio di conseguenza. Di fatto è un test, sebbene non strutturato formalmente. Osserviamo il seguente esempio: verifichiamo se la funzione `aggiungi` somma correttamente 2 e 3, confrontando il risultato con 5 (valore atteso).

```
def aggiungi(a, b):
    return a + b

# Test manuale semplice:
risultato = aggiungi(2, 3)
atteso = 5
if risultato == atteso:
    print("Test passato!")
else:
    print("Test fallito: atteso", atteso, "ma ottenuto", risultato)
```

(Il test sopra controlla solo un caso e segnala se passa o fallisce. Immagina di dover ripetere manualmente molti test: diventa noioso e soggetto a errori. Ecco perché esistono i framework di testing come unittest: aiutano a organizzare e automatizzare questi controlli.)

Testing: scrivere test con unittest

Il modulo `unittest` di Python fornisce classi e metodi per definire test in modo strutturato. L'idea base è creare una classe che estende `unittest.TestCase` e scrivere metodi al suo interno che inizino con `test_`: ognuno di questi è un caso di test. All'interno usiamo metodi di asserzione forniti da `TestCase` (come `self.assertEqual`, `self.assertTrue`, `self.assertRaises`, ecc.) per verificare le condizioni desiderate. Quando eseguiamo il test runner, verranno eseguiti tutti i metodi test e segnalati eventuali fallimenti.

Nel seguente esempio, definiamo una funzione `somma` e poi la classe di test `TestSomma` con un metodo `test_somma_numeri` che verifica che `somma(2,3)` restituisca 5. In fondo, la clausola `if __name__ == "__main__": unittest.main()` permette di eseguire i test lanciando semplicemente lo script (il test runner verrà avviato e cercherà automaticamente i metodi che iniziano con `test`):

```
import unittest

def somma(a, b):
    return a + b

class TestSomma(unittest.TestCase):
    def test_somma_numeri(self):
```

```
self.assertEqual(somma(2, 3), 5)

if __name__ == "__main__":
    unittest.main()
```

(Se salvi questo codice ad esempio in `test_somma.py` e lo esegui, vedrai l'output di un test eseguito: un punto `.` se passa, oppure dettagli dell'errore se fallisce. Puoi aggiungere altri metodi per testare altri casi, e `unittest` li eseguirà tutti. `unittest` supporta anche `setup/teardown` per preparare ambiente di test, raggruppare test suites, ecc., ma per ora il concetto chiave è: scrivere asserzioni in metodi di classe ereditata da `TestCase`.)

Testing: scrivere test con pytest

`pytest` è un framework di testing esterno (va installato con `pip install pytest`) molto popolare per la sua **semplicità**. Con `pytest` non c'è bisogno di definire classi `TestCase` o usare metodi specifici di un framework; basta scrivere funzioni Python il cui nome inizia con `test_` e usare semplici `assert` per le verifiche. `Pytest` troverà queste funzioni e le eseguirà, considerandole come test. Ha un output molto chiaro e potente in caso di fallimenti (mostra i valori esatti che hanno causato l'assertion failure).

Ecco un esempio comparabile al precedente, ma usando `pytest`. Creiamo una funzione da testare e una funzione di test. Invece di `self.assertEqual`, qui facciamo direttamente `assert somma(2,3) == 5`. Salvando questo in un file chiamato ad esempio `test_somma_pytest.py`, possiamo eseguire `pytest` nella cartella e verrà raccolto ed eseguito automaticamente:

```
# Per utilizzare pytest, installare con: pip install pytest

def somma(a, b):
    return a + b

def test_somma_numeri():
    assert somma(2, 3) == 5
```

(Lanciando `pytest`, vedremo in output quanti test sono stati raccolti e il risultato. `Pytest` gestisce anche situazioni più avanzate come `fixture` per predisporre ambiente, `parametri` per test parametrizzati, `integrazione` con altri plugin ecc. Ma la sua forza per iniziare è che i test sono semplicemente funzioni con `assert`, quindi molto facili da scrivere e leggere.)

Testing: organizzare i casi di test

Man mano che i test crescono di numero, è buona norma **organizzarli in modo strutturato**. Tipicamente si crea una directory dedicata (spesso chiamata `tests/`) dove mettere tutti i file di test. Ad esempio, se abbiamo un modulo `calcoli.py`, creeremo un file `test_calcoli.py` dentro `tests/` contenente i test per quel modulo. Questo aiuta a mantenere separato il codice di test da quello applicativo. I framework di testing come `pytest` e `unittest` hanno convenzioni per scoprire automaticamente i test in queste strutture (ad esempio, `pytest` cerca file che iniziano o finiscono con `test`).

Nel caso di `unittest`, possiamo anche organizzare i test in classi e moduli come parte di un pacchetto di test. L'importante è avere nomi coerenti e chiari. Di seguito, un esempio di come potrebbe apparire una struttura di progetto con una cartella per i test, parallela ai moduli dell'applicazione:

```
# Esempio di struttura di progetto con test:
progetto/
├── calcoli.py          # codice sorgente principale
├── tests/              # cartella per i test
│   ├── __init__.py
│   └── test_calcoli.py # file con test per calcoli.py
```

(In questa struttura, eseguendo `pytest` dalla root del progetto, verranno trovati ed eseguiti i test nel file `test_calcoli.py`. Anche con `unittest`, potremmo eseguire `python -m unittest discover` per far scoprire ed eseguire tutti i test nella cartella `tests`. Mantenere i test separati rende anche più facile escluderli dal pacchetto finale in fase di distribuzione.)

Testing: eseguire i test e leggere i risultati

Per eseguire i test scritti, a seconda del framework, usiamo comandi specifici. Con **unittest**, se abbiamo il blocco `unittest.main()` nei file di test, possiamo semplicemente eseguire quel file (`python test_mio.py`) e vedremo l'output dei test. In alternativa, `unittest` offre un comando di scoperta: `python -m unittest` (che di default cerca test in file che iniziano con `test` nell'albero). Con **pytest**, basta il comando `pytest` nella directory del progetto: questo troverà e farà partire tutti i test. Pytest inoltre produce un output più dettagliato su eventuali errori o assert falliti, evidenziando i valori.

Quando tutti i test passano, vedremo un report con esito positivo (in `unittest` spesso "OK", in `pytest` un elenco dei test eseguiti con `[100%]` e messaggio finale di successo). Se alcuni test falliscono, il report mostrerà i dettagli: quali test sono falliti e perché (aspettativa vs risultato). Ecco un esempio di output per un singolo test passato sia in `unittest` che in `pytest`:

```
# Eseguire i test con unittest (ricerca automatica):
# $ python -m unittest

# Output tipico:
# .
# -----
# Ran 1 test in 0.000s
# OK

# Eseguire i test con pytest:
# $ pytest

# Output tipico:
# ===== test session starts
# =====
# collected 1 item
#
# test_somma.py .
```



```
[100%]  
#  
# ===== 1 passed in 0.01s  
=====
```

(Nel caso di test falliti, l'output di pytest includerà il traceback del test e il valore atteso vs ottenuto, facilitando moltissimo il debugging del codice sotto test. Con unittest, vedremo l'AssertionError con eventuale messaggio e la traccia. Entrambi gli strumenti hanno opzioni per rendere l'output più verboso o silenzioso a piacimento.)

Performance: perché misurare le prestazioni

Oltre a far funzionare il codice correttamente, è importante considerare le **prestazioni** soprattutto per programmi che elaborano molti dati o operano in tempo reale. A volte un codice apparentemente giusto può essere troppo lento su grandi input. Invece di andare "a intuito", conviene **misurare** i tempi di esecuzione (profilare il codice) per identificare colli di bottiglia e sezioni critiche da ottimizzare.

Python offre vari strumenti per misurare le prestazioni: per misure veloci si può usare il modulo `time` per calcolare differenze temporali, oppure il modulo `timeit` per misurazioni precise ripetute. Per analisi più approfondite, c'è il modulo `cProfile` che profila l'intero programma e dice quanto tempo è speso in ogni funzione. Solo dopo aver misurato, ha senso ottimizzare (evitando la **premature optimization**).

Ad esempio, supponiamo di avere una funzione implementata in modo poco efficiente:

```
def somma_numeri(n):  
    risultato = 0  
    for i in range(n):  
        risultato += i  
    return risultato  
  
# Questa funzione somma tutti i numeri da 0 a n-1 in modo non ottimizzato  
(uso di loop Python)  
somma_numeri(1000000)
```

(Il codice sopra calcola la somma dei primi 1.000.000 di numeri con un ciclo Python, che potrebbe non essere velocissimo. Più avanti vedremo come misurare esattamente il tempo che impiega e confrontarlo con metodi alternativi.)

Performance: misurare il tempo con time

Il modo più semplice per misurare la durata di un'operazione in Python è usare il modulo `time`. In particolare, `time.time()` restituisce l'orario corrente in secondi (come timestamp). Salvando il tempo prima e dopo aver eseguito un certo blocco di codice, la differenza ci dà i secondi impiegati. Questo metodo è grezzo ma efficace per misurazioni ad esempio di funzione intera o di una porzione di script. Per maggiore precisione (soprattutto su tempi molto brevi) esiste `time.perf_counter()`, ma per iniziare `time.time()` va bene.

Nel codice seguente misuriamo il tempo per sommare un milione di numeri usando un loop, come definito nella funzione precedente. Prendiamo il tempo iniziale, eseguiamo l'operazione, poi prendiamo il tempo finale e stampiamo la differenza con 4 cifre decimali:

```
import time

start = time.time()
# Codice di cui misurare il tempo di esecuzione:
totale = 0
for i in range(1000000):
    totale += i
end = time.time()
print(f"Tempo impiegato: {end - start:.4f} secondi")
```

(Su computer moderni, la somma di 1e6 numeri potrebbe impiegare pochi centesimi di secondo o meno; spesso conviene aumentare un po' il lavoro (o iterare il test più volte) per ottenere misure più stabili. Ricorda di escludere dal calcolo di tempo eventuali operazioni I/O come stampe a video, che possono falsare la misura poiché più lente della computazione in memoria.)

Performance: misurare il tempo con `timeit`

Per misure più accurate e affidabili, Python fornisce il modulo `timeit`. Questo modulo esegue il codice da misurare molte volte (di default un milione di volte per piccole istruzioni, o un numero specificato da noi) e calcola il tempo totale o medio. Questo riduce l'incertezza dovuta ad altri processi del sistema, e fornisce risultati comparabili. Possiamo usare `timeit` tramite la funzione `timeit.timeit` oppure come linea di comando (`python -m timeit ...`).

Usare `timeit.timeit(stmt, number=N)` è semplice: `stmt` può essere una stringa con il codice da eseguire (attenzione: è eseguito in un contesto isolato, quindi importa eventuali moduli dentro la stringa stessa se servono), e `number` è quante volte eseguirlo. Ecco un esempio che confronta due modi di concatenare stringhe: con il metodo `join` e con una comprensione:

```
import timeit

tempo = timeit.timeit('"".join(str(n) for n in range(100))', number=10000)
print(f"Tempo totale per eseguire join 10000 volte: {tempo:.4f} secondi")
```

(Qui stiamo misurando 10.000 esecuzioni di un'operazione di join su 100 numeri. `timeit` restituisce il tempo totale. Possiamo variare `number` per adattarlo: se l'operazione è molto veloce, aumentare il numero di iterazioni migliora la precisione. `timeit` è utile anche per confrontare due approcci: potremmo eseguire due chiamate separate con codici diversi e confrontarne i tempi.)

Performance: profilare il codice con `cProfile`

Per capire **dove** il programma spende più tempo, soprattutto se è complesso, si utilizza il **profiling**. Il modulo `cProfile` (integrato) permette di eseguire un programma tracciando il tempo impiegato in ogni funzione chiamata. Alla fine produce un report con funzioni, numero di chiamate e tempo

cumulativo. Questo aiuta a individuare colli di bottiglia in funzioni specifiche. Possiamo usare `cProfile` programmaticamente o da riga di comando (`python -m cProfile script.py`).

Nel seguente esempio definiamo una funzione `calcola()` che fa un lavoro semplice ma iterativo, e usiamo `cProfile.run()` per eseguirla con profiling. Il risultato (stampato in console) elencherà la funzione `calcola` stessa e le funzioni interne Python chiamate (come `range` etc.), mostrando tempi e chiamate:

```
import cProfile

def calcola():
    somma = 0
    for i in range(10000):
        somma += i
    return somma

cProfile.run('calcola()')
```

(Il report di `cProfile` può essere lungo. Tipicamente, le colonne includono: numero di chiamate, tempo totale speso in quella funzione, tempo cumulativo (incluso il tempo nelle funzioni che essa chiama) e il nome della funzione. Per analisi più comode, si può salvare questo output e visualizzarlo con strumenti come `pstats` o librerie che generano grafici. Ma anche a colpo d'occhio, possiamo identificare quale funzione ha assorbito la maggior parte del tempo.)

Performance: ottimizzare il codice dopo la profilazione

Una volta identificati i punti critici, si può procedere a **ottimizzare** il codice. Le ottimizzazioni possono essere di vario tipo: usare algoritmi con complessità migliore, sfruttare funzioni built-in o librerie ottimizzate (spesso in C) al posto di loop Python lenti, ricorrere a strutture dati più adatte, o calcoli vettoriali con librerie come NumPy. A volte bastano piccole modifiche per grandi miglioramenti (ad esempio usare `' '.join(lista)` per concatenare stringhe invece di fare somme di stringhe in un loop, perché il join è implementato in C e molto efficiente).

È importante ribadire: **prima** misuriamo, **poi** ottimizziamo. Inutile complicare il codice per ottimizzare parti che non impattano significativamente sul tempo totale. Di seguito un esempio semplice di **confronto tra due metodi** e come un metodo risulti più performante: sommare numeri con un loop Python vs usare la funzione built-in `sum` (che è implementata in C e molto ottimizzata).

```
import time

# Approccio 1: somma con un ciclo for esplicito
start = time.time()
tot1 = 0
for i in range(1000000):
    tot1 += i
end = time.time()
print(f"Somma con loop: {end - start:.4f} s")
```

```
# Approccio 2: somma con la funzione built-in sum()
start = time.time()
tot2 = sum(range(1000000))
end = time.time()
print(f"Somma con sum(): {end - start:.4f} s")
```

(Su grandi numeri, in genere `sum()` risulta molto più veloce del loop manuale in Python, perché sposta il calcolo pesante a livello nativo. Il principio generale: sfruttare funzioni e librerie ottimizzate anziché reinventare ruote in puro Python. Inoltre, considerare complessità algoritmica: per elaborare N elementi, un algoritmo $O(N)$ sarà preferibile a uno $O(N^2)$ se N è grande. Il profiling può aiutarci a individuare quelle sezioni di codice da migliorare.)

AsyncIO: vantaggi della programmazione asincrona

Di default, Python esegue il codice in modo **sincrono**: una riga dopo l'altra, e se una operazione impiega tempo (ad esempio attesa di I/O come lettura da rete o disco), il programma resta bloccato finché non termina, per poi passare alla successiva. La **programmazione asincrona** mira a sfruttare meglio i tempi morti (come le attese di I/O) permettendo di fare altro nel frattempo. In Python, la libreria `asyncio` consente di scrivere codice concorrente usando lo stesso thread (quindi senza complicazioni di threading) attraverso l'uso di *coroutine* e del ciclo eventi (event loop).

In pratica, con AsyncIO possiamo lanciare più operazioni (coroutine) "in parallelo" (in realtà vengono gestite cooperativamente dall'event loop: quando una coroutine è in attesa, l'event loop passa a eseguirne un'altra). Questo risulta in un enorme vantaggio in scenari **I/O-bound** (come molte chiamate web, accesso a database, pause temporali) perché possiamo far progredire altre operazioni durante i tempi di attesa, invece di tenere la CPU inattiva. Per contro, su compiti pesantemente **CPU-bound** AsyncIO non dà benefici, poiché una singola coroutine affamerebbe l'interprete finché non cede il controllo.

Consideriamo come esempio di base due chiamate a una funzione che effettua una pausa (simulando un'attesa I/O). Sincronamente, le attese si sommano, rendendo il tempo totale circa la somma dei singoli, mentre asincronamente potremmo ridurre il tempo totale gestendole in overlap. Ecco un esempio sincrono (sequenziale):

```
import time

def saluta(nome):
    time.sleep(1)
    print(f"Ciao {nome}!")

# Chiamate sincrone (una dopo l'altra)
saluta("Alice")
saluta("Bob")

# Questo approccio impiega ~2 secondi totali per salutare due persone (1s ciascuna in sequenza).
```

AsyncIO: definire coroutines con `async/await`

In AsyncIO, definiamo le funzioni "asincrone" usando `async def`. Queste funzioni, quando chiamate, non eseguono subito il loro corpo, ma restituiscono un oggetto **coroutine**. Per eseguire la coroutine dobbiamo o **attenderla** con `await` all'interno di un'altra coroutine, oppure passarla all'event loop. Dentro una coroutine possiamo chiamare (aspettare) altre coroutine usando appunto `await`. Ad esempio, `await asyncio.sleep(1)` sospende la coroutine corrente per 1 secondo, permettendo ad altre di avanzare nel frattempo, e riprende dopo l'attesa.

Per avviare effettivamente il loop asincrono, fuori dalle coroutine usiamo `asyncio.run(coroutine_principale())` (in passato si usava creare un loop e chiamare `run_until_complete`). Vediamo un esempio semplice di funzioni asincrone: una coroutine `saluta` che attende un secondo e poi stampa un saluto, e una coroutine principale `main` che le chiama in sequenza usando `await`. Notare che in questo caso, anche se usiamo `async/await`, stiamo ancora eseguendo in modo sequenziale (prima Alice poi Bob) perché le attendiamo uno dopo l'altro.

```
import asyncio

async def saluta(nome):
    await asyncio.sleep(1) # simula un'attesa (es. operazione I/O)
    print(f"Ciao {nome}!")

async def main():
    print("Inizio")
    await saluta("Alice") # chiamata sequenziale: attende che saluta finisca
    await saluta("Bob")
    print("Fine")

asyncio.run(main())
```

(Se esegui questo codice, vedrai che impiega ~2 secondi totali e stampa "Inizio", dopo 1s "Ciao Alice!", dopo un altro secondo "Ciao Bob!" e infine "Fine". L'utilità qui non è tanto la concorrenza (che vediamo nel prossimo step) ma la struttura: abbiamo definito funzioni asincrone che possiamo comporre con `await`. Questo prepara il terreno per eseguirne più di una contemporaneamente.)

AsyncIO: eseguire coroutines in parallelo

Il vero vantaggio di AsyncIO si vede facendo partire più coroutines "in parallelo" (concorrenza). Per farlo, invece di `await`arle immediatamente una dopo l'altra, possiamo creare dei **Task** con `asyncio.create_task(coroutine)`. Creare un task significa dire all'event loop "inizia a eseguire questa coroutine, ma non aspettare subito il suo risultato, prosegui intanto". Il task viene schedato e girerà sullo sfondo asincrono finché non finisce. Noi possiamo quindi avviare più task quasi contemporaneamente e poi usare `await` su di essi successivamente (o usare funzioni come `asyncio.gather` che vedremo dopo).

Riprendendo l'esempio precedente, modifichiamo `main` per lanciare le due coroutines di saluto *quasi insieme* usando `create_task`. In questo modo entrambe inizieranno la loro attesa di 1 secondo quasi nello stesso momento, e l'event loop le porterà avanti entrambe. Il tempo totale per eseguire entrambi i saluti sarà di circa 1 secondo (invece dei 2 sequenziali). Ecco come fare:

```

import asyncio

async def saluta(nome):
    await asyncio.sleep(1)
    print(f"Ciao {nome}!")

async def main():
    # Avvia due coroutines in parallelo senza aspettarle subito
    task1 = asyncio.create_task(saluta("Alice"))
    task2 = asyncio.create_task(saluta("Bob"))
    # (qui entrambe le coroutines sono in esecuzione concorrente)
    await task1 # aspetta il completamento della prima coroutine
    await task2 # aspetta il completamento della seconda

asyncio.run(main())

```

(In output vedremo "Ciao Alice!" e "Ciao Bob!" apparire praticamente nello stesso momento dopo circa 1 secondo. L'ordine preciso potrebbe dipendere da come il scheduler gestisce le due coroutines, ma entrambe finiranno dopo ~1s. Questo dimostra come AsyncIO sfrutta l'attesa di una coroutine per mandare avanti l'altra. Il nostro event loop non rimane mai inattivo durante quelle attese, a differenza del caso sincrono.)

AsyncIO: sincronizzare coroutines con `asyncio.gather`

Un modo conveniente per eseguire più coroutines in parallelo e aspettare i loro risultati è usare `asyncio.gather`. Questa funzione accetta come argomenti (o lista) varie coroutines e restituisce un'unica coroutine che termina quando tutte quelle date sono concluse, restituendo i risultati in una tupla o lista. È quindi un equivalente di alto livello rispetto a creare tasks manualmente e poi `await` su ciascuno: `await asyncio.gather(A, B, C)` lancia A, B, C in parallelo e attende che *tutte* finiscano, restituendo `[risultatoA, risultatoB, risultatoC]`.

Nell'esempio seguente, supponiamo di avere una coroutine `quadrato(x)` che calcola il quadrato di un numero dopo un breve ritardo (simulando un calcolo asincrono). Usiamo `asyncio.gather` per lanciare più istanze di `quadrato` contemporaneamente e raccogliere i loro risultati una volta pronte. Il vantaggio è che scriviamo un solo `await` e otteniamo direttamente la lista dei risultati.

```

import asyncio

async def quadrato(x):
    await asyncio.sleep(1)
    return x * x

async def main():
    risultati = await asyncio.gather(
        quadrato(2),
        quadrato(3),
        quadrato(4)
    )
    print("Risultati:", risultati)

```

```
asyncio.run(main())
```

(In questo caso, ogni quadrato impiega ~1 secondo "di attesa". L'uso di `gather` permette di eseguire i tre in parallelo, impiegando circa 1 secondo totale invece di 3. Alla fine `risultati` conterrà `[4, 9, 16]`. `asyncio.gather` è molto utile quando bisogna lanciare batch di operazioni asincrone e poi usarne tutti i risultati. Se una di queste fallisce con eccezione, per default `gather` propagherà l'errore, ma esistono opzioni per gestire anche errori parziali.)

AsyncIO: esempio pratico di uso asincrono

Facciamo un esempio pratico: immaginiamo di dover ottenere dati da più fonti (per esempio chiamate HTTP a diverse API). Con AsyncIO possiamo avviare tutte le richieste "insieme" e attendere che tutte rispondano, riducendo drasticamente il tempo totale rispetto a farle in sequenza. Simuliamo questa situazione con una coroutine `get_data` che attende un tempo random (come se fosse il tempo di risposta di un servizio esterno) e poi stampa un messaggio. Nel `main` lanciamo più `get_data` concorrenti e aspettiamo che tutte completino con `gather`. Noteremo che i messaggi possono arrivare in ordine diverso (dipendendo dal ritardo random), ma il totale sarà circa il massimo dei singoli ritardi invece che la somma.

```
import asyncio, random

async def get_data(fonte):
    # Simula un tempo di risposta variabile tra 0.5 e 2.0 secondi
    await asyncio.sleep(random.uniform(0.5, 2.0))
    print(f"Dati ricevuti da fonte {fonte}")

async def main():
    tasks = [asyncio.create_task(get_data(n)) for n in range(1, 4)]
    await asyncio.gather(*tasks)

asyncio.run(main())
```

(Eseguendo questo codice, per esempio, potrebbe stampare:

```
Dati ricevuti da fonte 2
Dati ricevuti da fonte 1
Dati ricevuti da fonte 3
```

in un ordine qualsiasi, dopo un tempo totale vicino a 2.0 secondi (il più lento dei tre). Sincronamente avrebbe impiegato la somma ~> 4-5 secondi. Questo esempio illustra chiaramente come AsyncIO consente di sfruttare al meglio le attese I/O.)

AsyncIO: quando (non) usare AsyncIO

È importante capire **quando AsyncIO è la scelta giusta**. Come accennato, AsyncIO rende molto efficiente la gestione di molte operazioni I/O-bound concorrenti (rete, file, sleep, ecc.) in un solo thread,

evitando il sovraccarico del threading tradizionale e aggirando in parte il GIL per I/O. Tuttavia, **per operazioni CPU-intensive** (calcoli pesanti, elaborazioni numeriche lunghe) AsyncIO non offre benefici, perché una coroutine che esegue calcolo puro monopolizzerà l'event loop finché non cede il controllo (ad esempio con un `await`). In quei casi, per parallelizzare bisogna ricorrere a **thread** o **processi multipli** (modulo `threading` o `multiprocessing`), sfruttando più core.

Riassumendo: usa AsyncIO quando hai molte attese (socket, HTTP, query DB, ecc.), non usarlo aspettandoti accelerazione in calcolo matematico puro. In scenari reali, spesso si combinano le tecniche: ad esempio AsyncIO per gestire migliaia di connessioni client in un server web (I/O-bound), ma delegando eventuali compiti gravosi a thread separati o a processi worker.

```
# AsyncIO è indicato per operazioni I/O-bound (es: chiamate di rete, lettura/
scrittura file),
# mentre per calcoli CPU-bound conviene usare thread o processi separati.
# Esempio (indicativo): lanciare un calcolo intensivo su un thread:
import threading

def conta_fino_a(n):
    while n > 0:
        n -= 1

# Avvia due thread per svolgere calcoli in parallelo (CPU-bound)
t1 = threading.Thread(target=conta_fino_a, args=(10000000,))
t2 = threading.Thread(target=conta_fino_a, args=(10000000,))
t1.start(); t2.start()
t1.join(); t2.join()
print("Terminato calcolo su due thread")
```

(L'esempio sopra crea due thread per eseguire una funzione CPU-bound in parallelo. Questo può sfruttare macchine multi-core (anche se in Python c'è il GIL, due thread su pura CPU non daranno un reale parallelismo su core diversi; per quello servirebbero i processi). Il messaggio chiave è: valutare la natura del problema. AsyncIO è formidabile per I/O (si pensi ad un server che gestisce migliaia di socket concorrenti), ma inutile per velocizzare loop computazionali; per quelli meglio ricorrere a altre strategie.)

Packaging: moduli vs pacchetti Python

Quando un progetto Python cresce oltre un singolo script, conviene organizzarlo in più **moduli** e **pacchetti**. Un *modulo* in Python è semplicemente un file `.py` (che può contenere funzioni, classi, ecc.), importabile tramite il suo nome di file. Un *pacchetto* è invece una cartella contenente un file `__init__.py` (anche vuoto) e uno o più moduli: in pratica il pacchetto raggruppa logicamente più moduli sotto un namespace comune (il nome della cartella). Ad esempio, potremmo avere un pacchetto `utilita` con dentro vari moduli (`stringhe.py`, `numeri.py`, ecc.), e importarli con `import utilita.stringhe`.

Capire la differenza è importante quando si parla di *packaging* di un progetto. "Pacchettizzare" un progetto significa prepararlo per la distribuzione/installazione, e in genere consiste nel fornire i file Python (organizzati in moduli/pacchetti) insieme a metadata come il nome del progetto, versione, dipendenze, etc., in un formato che può essere installato con `pip`. Prima di vedere gli strumenti di

packaging (setup.py, pyproject.toml), è utile avere chiara la struttura del progetto in termini di moduli e pacchetti:

```
# Esempio:
# modulo singolo:
#   utilita.py    (un singolo file Python che è un modulo)

# pacchetto:
#   utilita/      (cartella pacchetto)
#       __init__.py (rende la cartella un pacchetto importabile)
#       stringhe.py (modulo all'interno del pacchetto)
#       numeri.py  (altro modulo nel pacchetto)
```

(In generale, se il tuo progetto è composto da più file .py correlati, conviene farne un pacchetto. Ad esempio un progetto "Mioprogetto" potrebbe contenere una cartella `mioprogetto/` con dentro `__init__.py` e vari moduli. Questo pacchetto a sua volta verrà distribuito.)

Packaging: struttura di un progetto Python

Vediamo una tipica **struttura di progetto** Python pronta per il packaging e la distribuzione. Avremo: una cartella principale col nome del progetto o simile, dentro di essa il pacchetto Python con il codice, e alcuni file di configurazione. Tradizionalmente si usava un file `setup.py` per definire le informazioni di packaging. Recentemente, con le nuove specifiche (PEP 517/518), si può usare `pyproject.toml` per configurare il packaging e build system. Spesso si include anche un file `README.md` come documentazione, ed eventualmente un `setup.cfg` o altri file se necessari.

Ecco la struttura base di un progetto chiamato ad esempio "miopacchetto":

miopacchetto/	# directory radice del progetto
├── miopacchetto/	# pacchetto Python principale con il codice
│ ├── __init__.py	# rende la cartella un pacchetto Python
│ └── modulo.py	# un modulo con funzioni/classi del progetto
├── README.md	# descrizione del progetto
├── setup.py	# script di setup (metodo tradizionale di packaging)
└── pyproject.toml	# configurazione del progetto (nuovo standard)

(In questa struttura, il nome del pacchetto Python `miopacchetto` è uguale al nome del progetto per semplicità, ma non è obbligatorio. `setup.py` e `pyproject.toml` possono coesistere se stiamo migrando al nuovo sistema. `README.md` sarà usato per la pagina PyPI del progetto. Spesso troviamo anche un `LICENSE` file, e cartella `tests/` per i test come visto prima, ma li omettiamo qui per brevità.)

Packaging: il file setup.py

`setup.py` è (o era) il cuore del packaging Python tradizionale usando **setuptools** o **distutils**. Si tratta di uno script Python che chiama la funzione `setuptools.setup()` fornendo tutti i parametri

(metadata e opzioni) che descrivono come costruire il pacchetto. Ad esempio: nome del pacchetto, versione, lista di pacchetti o moduli da includere, eventuali dipendenze (`install_requires`), informazioni sull'autore, licenza, ecc. Quando un utente esegue `pip install .` nella directory del progetto, pip cerca `setup.py` e lo esegue per installare il pacchetto.

Un esempio minimo di `setup.py` usando `setuptools` potrebbe essere:

```
import setuptools

setuptools.setup(
    name="miopacchetto",
    version="0.1.0",
    packages=["miopacchetto"],      # pacchetti da includere
    install_requires=[],            # altre librerie richieste, es: ["requests"]
)
```

(In un `setup.py` reale si aggiungerebbero molti parametri: `description`, `author`, `author_email`, `url`, `license`, `long_description` (spesso letto da `README`), `classifiers` (per categorizzare su PyPI), ecc. Qui però ci concentriamo sull'essenziale. Da notare: `packages=["miopacchetto"]` indica di includere il pacchetto principale. `Setuptools` troverà automaticamente i moduli dentro di esso. Se avessimo sottopacchetti, potremmo usare `setuptools.find_packages()` per scoprirli automaticamente.)

Packaging: il file `pyproject.toml`

Il nuovo metodo (PEP 518 e successivi) per definire la configurazione di build di un progetto Python è tramite `pyproject.toml`. Questo file in formato TOML dichiara quale *build system* usare (ad esempio `setuptools`, `flit`, `poetry`) e può contenere i metadata del progetto (PEP 621). In pratica, `pyproject.toml` può sostituire `setup.py` e `setup.cfg` centralizzando la configurazione. Ad esempio, usando `setuptools` come backend classico, possiamo specificare le informazioni di progetto nella sezione `[project]`.

Ecco un esempio minimale di `pyproject.toml` per il nostro progetto, che indica di usare `setuptools` e fornisce i metadata base:

```
[build-system]
requires = ["setuptools", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "miopacchetto"
version = "0.1.0"
description = "Esempio di pacchetto Python"
authors = [
    { name="Mario Rossi", email="mario.rossi@example.com" }
]
dependencies = []
```

(In questo TOML, la sezione `[build-system]` dice a `pip` di usare `setuptools` come backend di build (e di assicurarsi che `setuptools` e `wheel` siano installati per costruire il pacchetto). La sezione `[project]` segue la specifica PEP 621: possiamo elencare nome, versione, descrizione, autori, dipendenze, ecc. Usando questo metodo, non è necessario avere un `setup.py` esplicito con codice eseguibile - molti dettagli sono dichiarativi. Al comando `pip install .`, `pip` leggerà `pyproject.toml` e procederà di conseguenza.)

Deployment: installare pacchetti con pip

Una volta creato un pacchetto (localmente, o pubblicato su PyPI), **installarlo** in un ambiente è questione di un semplice comando `pip`. `pip` è il gestore di pacchetti per Python: scarica i pacchetti da **PyPI (Python Package Index)** o li installa da percorsi locali. Per esempio, se abbiamo pubblicato "miopacchetto" su PyPI, chi vuole usarlo può fare: `pip install miopacchetto`. Questo comando scaricherà la versione più recente disponibile e installerà il pacchetto (e le sue dipendenze eventualmente).

Durante lo sviluppo, potremmo voler installare il pacchetto localmente (ad esempio per testarlo in un altro progetto). Se siamo nella directory del progetto con `setup.py` o `pyproject.toml`, eseguire `pip install .` installerà il pacchetto presente in quella directory nel nostro ambiente corrente. Esiste anche l'opzione `pip install -e .` (editable mode) per installarlo in modo modificabile (i link simbolici vengono creati, così possiamo editare i file e avere subito gli effetti senza reinstallare).

Alcuni esempi di comandi pip per chiarezza:

```
# Installare un pacchetto dalla repository PyPI:
pip install requests

# Installare il pacchetto corrente (nella dir corrente) nel proprio ambiente:
pip install .
```

(Il secondo comando legge le info di packaging (`setup.py/pyproject`) e copia i file del pacchetto nelle site-packages dell'ambiente. Questo è utile se ad esempio stiamo lavorando su un progetto e vogliamo usare le sue funzionalità in un altro script esterno, possiamo installarlo localmente prima di pubblicarlo.)

Deployment: pubblicare un pacchetto su PyPI

Dopo aver preparato il pacchetto e testato che sia installabile, potremmo volerlo **pubblicare su PyPI** per condividerlo con altri (o semplicemente per distribuirlo facilmente tra i nostri ambienti). PyPI è il repository pubblico dove vivono la maggior parte delle librerie Python. Per pubblicare, prima bisogna registrare un account su PyPI. Poi occorre **costruire i pacchetti distributivi** del progetto: tipicamente un *sdist* (source distribution, un tar.gz con i sorgenti) e uno o più *wheel* (binary distribution, pacchetti precompilati per specifiche piattaforme, se applicabile).

Con `setuptools` e `wheel` installati, il modo più semplice oggi è usare il modulo `build`: `python3 -m build` dalla directory del progetto creerà i file in una cartella `dist/`. Quindi, per caricarli su PyPI si usa `twine`, un tool che trasferisce i file su PyPI in modo sicuro. In sintesi:

```
# Generare i pacchetti (source e wheel) del progetto:
python3 -m build
```

```
# Caricare i pacchetti generati su PyPI:
python3 -m twine upload dist/*
```

(Durante l'upload, twine chiederà le credenziali PyPI. Dopo il successo, chiunque potrà fare `pip install miopacchetto` e pip scaricherà proprio quei file. È buona pratica provare prima l'upload su TestPyPI, un ambiente di test, per verificare che tutto funzioni. Una volta su PyPI, il tuo pacchetto è pubblico con la versione specificata.)

```
# Preparazione del pacchetto (da terminale):
# $ python3 -m build
#
# Pubblicazione su PyPI (richiede account PyPI configurato):
# $ python3 -m twine upload dist/*
```

Deployment: creare un eseguibile con PyInstaller

A volte si vuol distribuire un programma Python a utenti finali che non hanno Python installato o comunque semplificare la consegna. **PyInstaller** è uno strumento che prende uno script (o un progetto) Python e genera un **eseguibile standalone** che include l'interprete Python e tutte le dipendenze. In questo modo, l'utente può eseguire il programma senza installare nulla (l'eseguibile è però specifico per il sistema operativo su cui è creato).

Usare PyInstaller è semplice: dopo averlo installato (`pip install pyinstaller`), si può eseguire: `pyinstaller --onefile mio_script.py`. L'opzione `--onefile` fa sì che tutto venga messo in un unico file eseguibile (altrimenti crea una cartella con exe e librerie). PyInstaller analizzerà le importazioni, includerà le librerie necessarie e produrrà l'exe nella cartella `dist/`. Bisogna testare che funzioni (a volte servono opzioni aggiuntive se qualche modulo dinamico non viene rilevato automaticamente).

Esempio di utilizzo PyInstaller:

```
# Creare un eseguibile standalone (un file) per uno script Python:
pyinstaller --onefile mia_app.py

# (Su Windows genererà ad es. mia_app.exe in dist/, su Linux un binario ELF)
```

(L'eseguibile generato tende ad essere piuttosto grande perché include Python stesso e tutte le librerie. Tuttavia, è una soluzione ottima per distribuire utility o applicazioni GUI a utenti non tecnici. Esistono alternative come `cx_Freeze`, `Nuitka`, etc., ma PyInstaller è tra i più usati.)

Deployment: creare un archivio eseguibile con zipapp

Python offre anche un meccanismo integrato per creare **archivi eseguibili** con il modulo `zipapp`. In pratica si può impacchettare un intero programma Python (file .py e pacchetti) in un unico file con estensione `.pyz` (che è essenzialmente uno zip contenente i file). Questo archivio include al suo interno uno shebang e un piccolo bootstrap in modo che, se lanciato con Python, venga eseguito come

programma. La differenza con PyInstaller è che il `.pyz` richiede comunque che l'utente abbia Python installato sulla macchina (della giusta versione). È utile per distribuire applicazioni Python in formato singolo file senza compilare binari.

Per creare un archivio eseguibile, si può usare `zipapp.create_archive(source_dir, target='app.pyz', main='module:func')`. Il parametro `main` specifica qual è il punto di ingresso (funzione principale da eseguire). Se non lo si specifica, Python cercherà un `if __name__ == "__main__"` come al solito.

Ecco un esempio dove creiamo un archivio `miopacchetto.pyz` dalla cartella del progetto. Una volta creato, si può eseguire con `python miopacchetto.pyz` (o renderlo eseguibile con shebang).

```
import zipapp

# Crea un archivio eseguibile 'miopacchetto.pyz' dalla cartella
'miopacchetto'
zipapp.create_archive('miopacchetto', target='miopacchetto.pyz')
```

(Ricorda: il modulo/funzione principale per l'esecuzione deve essere specificato. Nel nostro caso, se `miopacchetto/__main__.py` esiste, quello verrà usato automaticamente come entry point. Altrimenti potremmo passare `main="miopacchetto.modulo:main"` se abbiamo una funzione `main()` definita. Questo approccio di `zipapp` è meno comune, ma molto interessante per shareware interni e tool distribuiti in ambito controllato dove si può assumere la presenza di Python.)

Deployment: gestire ambienti virtuali (venv)

Quando si parla di deployment (soprattutto in ambito di sviluppo e test), non si può non menzionare gli **ambienti virtuali**. Un ambiente virtuale Python (creato con `venv` o altri tool come `virtualenv`, `conda`, etc.) è una copia isolata dell'interprete Python in cui possiamo installare pacchetti senza influenzare il sistema globale. Ciò è fondamentale per evitare conflitti di dipendenze e per replicare ambienti puliti. Ad esempio, per provare l'installazione del nostro pacchetto, possiamo creare un nuovo `venv`, attivarlo, e poi fare `pip install`.

Come creare e attivare un `venv` standard:

```
# Creazione di un nuovo ambiente virtuale chiamato 'venv':
python3 -m venv venv

# Attivazione su Linux/macOS:
source venv/bin/activate

# (Su Windows, usare:)
venv\Scripts\activate.bat
```

(Dopo l'attivazione, il prompt del terminale spesso cambia per indicare che si è dentro l'ambiente. Da lì, ogni `pip install` andrà nell'ambiente virtuale, mantenendo pulito il sistema. Per uscire dall'ambiente, basta eseguire `deactivate`. Utilizzare ambienti virtuali è considerata una best practice nello sviluppo Python)

moderno, sia per testare il deployment, sia per lavorare a progetti diversi con dipendenze diverse senza interferenze.)

Risorse utili

Abbiamo coperto molti argomenti. Per approfondire e consultare dettagli specifici, ecco alcune **risorse utili**: la **documentazione ufficiale Python** è sempre un riferimento prezioso. Ad esempio, la documentazione del modulo logging spiega in dettaglio tutte le opzioni e configurazioni ¹; la sezione su unittest copre ulteriori funzionalità del framework di test ²; per AsyncIO, c'è un tutorial concettuale oltre alla reference ³. Inoltre, il **Python Packaging User Guide** offre guide passo-passo su packaging e distribuzione ⁴, ed è aggiornato con le best practice più recenti.

Oltre ai manuali ufficiali, esistono ottimi tutorial e articoli online: ad esempio *Real Python* e *Docs.python.org HOWTOs* per logging e asyncio, o il *pytest documentation* per imparare rapidamente pytest. In generale, quando esplori uno di questi temi, cerca la documentazione ufficiale e verifica eventuali aggiornamenti (soprattutto AsyncIO e packaging hanno evoluto le pratiche negli ultimi anni).

```
# Documentazione Python e guide:
# - logging: https://docs.python.org/3/library/logging.html
# - unittest: https://docs.python.org/3/library/unittest.html
# - asyncio: https://docs.python.org/3/library/asyncio.html
# - Packaging Python Projects: https://packaging.python.org/en/latest/tutorials/packaging-projects/
# - PyPI (guida): https://packaging.python.org/en/latest/tutorials/packaging-projects/#uploading-to-pypi
```

Altri strumenti utili

Nel panorama Python esistono molti strumenti avanzati correlati agli argomenti che abbiamo visto, che potresti esplorare in futuro per esigenze specifiche:

- **Scheduling avanzato**: se `schedule` non basta, c'è `APScheduler` (Advanced Python Scheduler) che consente di schedulare job con più flessibilità (anche con schedulazioni tipo cron) all'interno di applicazioni Python, e `Celery` per distribuire task asincroni su più macchine usando code di messaggi (ottimo per carichi di lavoro differiti o periodici su larga scala).
- **Logging aggiuntivo**: la libreria `loguru` fornisce un logging facile da usare con molte funzionalità out-of-the-box (colori, rotazione file, ecc.) senza configurazione complicata.
- **Debugging e sviluppo**: strumenti come `ipdb` (versione di pdb integrata con IPython) offrono un'esperienza di debug migliorata. Inoltre, usare un IDE con debugger grafico (PyCharm, VSCode) può velocizzare molto il ciclo di debug.
- **Testing avanzato**: oltre a pytest (già menzionato), c'è `hypothesis` per *property-based testing* (genera automaticamente input casuali per scovare edge cases), molto potente per testare funzioni puramente logiche. `tox` è un tool che automatizza i test su più ambienti (eseguendo i test su diverse versioni di Python in isolamento).
- **Profiling e performance**: per profilare l'uso di memoria ci sono librerie come `memory_profiler`, o per profilare riga-per-riga il tempo c'è `line_profiler`. Per performance critiche, si può valutare l'uso di Cython o di librerie in C/C++ per parti del codice.

- **Packaging e gestione dipendenze:** strumenti come **Poetry** e **Pipenv** forniscono un'esperienza più integrata per gestire dipendenze, virtual env e packaging, semplificando molto la pubblicazione di pacchetti.
- **Deployment e container:** in ambito deployment di applicazioni complete, **Docker** è spesso usato per creare container con tutto l'ambiente Python + applicazione preconfigurato, assicurando che l'app giri nello stesso modo ovunque.

```
# Altri strumenti da esplorare:
# - APScheduler (schedulazione avanzata di job, anche cron-like all'interno
di app Python)
# - Celery (task queue per esecuzioni asincrone e distribuite su più nodi,
con broker tipo Redis/RabbitMQ)
# - loguru (libreria di logging alternativa con API semplificata)
# - ipdb (estensione interattiva di pdb per debugging, integrata con IPython)
# - Hypothesis (testing basato su proprietà: genera casi di test
automaticamente)
# - tox (automatizza test su ambienti multipli, utile per librerie supportate
su varie versioni Python)
# - Poetry / Pipenv (tool moderni per gestire dipendenze e packaging in modo
dichiarativo e isolato)
# - Docker (containerizzazione dell'applicazione per deployment replicabile
su server)
```

Esercizio: script di backup automatizzato

Scopo: Mettere in pratica automazione file e logging. Prova a creare uno **script Python** che effettui il backup di tutti i file con estensione `.txt` da una cartella di origine a una cartella di destinazione. In particolare lo script dovrebbe: creare la cartella di backup se non esiste, copiare tutti i file `.txt` dalla cartella origine a quella di destinazione mantenendo il nome, e registrare ogni azione. Inizialmente puoi semplicemente stampare su console i nomi dei file copiati; se vuoi, estendi lo script usando il modulo `logging` per scrivere le stesse informazioni su un file di log.

Suggerimenti:

1. Usa `pathlib.Path` per rappresentare le cartelle di origine e destinazione e per cercare i file `.txt` (metodo `.glob("*.txt")`).
2. Usa `Path.mkdir(exist_ok=True)` per creare la cartella di destinazione se non c'è.
3. Usa `shutil.copy` per copiare ciascun file trovato nella nuova cartella.
4. Per ciascun file copiato, stampa un messaggio (o logga con logging) del tipo "Copiato <nomefile> in backup/".

```
# Suggerimento di struttura:
from pathlib import Path
import shutil
import logging

origine = Path("cartella_origine")
dest = Path("cartella_backup")
dest.mkdir(exist_ok=True)
for file in origine.glob("*.txt"):
```

```
shutil.copy(file, dest / file.name)
print(f"Copiato {file.name} in {dest}/")
# logging.info(f"Copiato {file.name} in backup")
```

(Dopo aver scritto lo script, testalo creando qualche file di testo di esempio. Esecuzioni successive dovrebbero sovrascrivere i file di backup esistenti; potresti migliorare lo script aggiungendo timestamp al nome del backup o organizzando in sottocartelle per date, ma questo esula dall'esercizio base.)

Esercizio: testare e correggere una funzione

Scopo: Applicare i concetti di testing unitario e debugging. Ti viene fornita la seguente funzione con un bug noto:

```
def somma(a, b):
    return a - b # BUG: dovrebbe usare + invece di -
```

1. Scrivi un caso di **test** (usando `unittest` o `pytest`) che verifica la correttezza di `somma`. Ad esempio, controlla che `somma(5, 3)` restituisca `8`. Il test attualmente fallirà a causa del bug.
2. Esegui il test per confermare il fallimento (se usi `unittest`, vedrai un `AssertionError`; con `pytest` vedrai direttamente dove l'assert è diverso).
3. Correggi la funzione `somma` per farla passare (ovvero, implementa l'addizione correttamente) e riesegui il test assicurandoti che ora passi.

Suggerimenti: Se usi `unittest`, crea una classe `TestSomma` con metodo `test_somma_base` che fa `self.assertEqual(somma(5,3), 8)`. Se usi `pytest`, basta una funzione `test_somma_base()` con `assert somma(5,3) == 8`.

```
# Data la funzione (con bug) seguente:
def somma(a, b):
    return a - b # BUG: dovrebbe sommare a + b

# 1. Scrivere un test (unittest o pytest) che verifica somma(5, 3) == 8.
# 2. Eseguire il test: ci si aspetta un fallimento.
# 3. Correggere la funzione somma (ritornando a + b) e verificare che il test passi.
```

(Questo esercizio ti guida nel ciclo TDD (Test-Driven Development) basilare: scrivi un test che cattura il comportamento desiderato, vedi il test fallire sul codice bug, aggiusta il codice e infine il test passa. È un ottimo modo per validare le tue funzioni man mano che le sviluppi.)

Esercizio: confronto di performance tra soluzioni

Scopo: Utilizzare gli strumenti di misurazione delle prestazioni per confrontare due approcci diversi a un problema. Un esempio classico è creare una lista con il risultato di un calcolo su una serie di elementi. Ti proponiamo di confrontare:

- **Metodo A:** usare un ciclo `for` per aggiungere elementi a una lista (es: `append(i*i)` per `i` da 0 a `N`).
- **Metodo B:** usare una **list comprehension** equivalente (`[i*i for i in range(N)]`).

Procedi così: scegli un N sufficientemente grande (es. $N = 1000000$). Misura il tempo impiegato dal Metodo A e dal Metodo B per creare la lista dei quadrati da 0 a N-1. Puoi usare `time.time()` prima e dopo ciascun metodo per avere tempi grossolani. In alternativa, usa `timeit` per eseguire magari 10 volte ogni metodo e ottenere un tempo medio. Stampa i risultati e determina quale metodo risulta più veloce in Python.

Suggerimenti:

- Ricorda di inizializzare le strutture dati fuori dal timing se non vuoi includere quel tempo (nel nostro caso, creare una lista vuota vs usare comprehension ha differenze intrinseche).
- Se usi `timeit.timeit`, puoi passare come stmt la stringa di comprehension e come setup magari un import di math se calcoli cose matematiche (non necessario per quadrato).
- In genere, le comprensioni e le funzioni built-in sono ottimizzate in C e superano i loop espliciti in Python. Questo esercizio dovrebbe confermarlo.

```
# Esempio: confrontare due metodi per generare una lista di quadrati fino a N-1
import time

N = 1000000

# Metodo 1: loop for con append
start = time.time()
quadrati1 = []
for i in range(N):
    quadrati1.append(i * i)
end = time.time()
print(f"Loop for: {end - start:.4f} s")

# Metodo 2: list comprehension
start = time.time()
quadrati2 = [i * i for i in range(N)]
end = time.time()
print(f"List comprehension: {end - start:.4f} s")

# (Opzionale: verificare che quadrati1 e quadrati2 siano identiche per correttezza)
```

(Aspettati di vedere la list comprehension più veloce del loop in Python puro. In alcune prove, il guadagno può essere significativo. Se vuoi sperimentare, potresti anche confrontare con una soluzione che utilizza NumPy: `np.arange(N)**2`, che sposta il calcolo a livello C vettoriale e di solito è ancora più veloce. Questo ci insegna che scegliere gli strumenti giusti può dare enormi benefici in termini di performance.)

Conclusioni

In questa lezione introduttiva abbiamo esplorato diversi aspetti avanzati dell'uso di Python: dall'automazione di compiti di sistema, al logging per monitorare l'esecuzione, alle tecniche di debugging e testing per migliorare la qualità del codice, fino alla misurazione delle prestazioni e alla programmazione asincrona per scrivere programmi più efficienti e responsivi. Abbiamo anche visto come preparare e distribuire un progetto Python, sia attraverso pacchetti installabili con pip, sia come

eseguibili per utenti finali. Questi strumenti e concetti sono fondamentali per andare oltre i semplici script e sviluppare software di livello superiore in Python.

Il consiglio finale è di **praticare** quanto appreso: automatizza una piccola attività quotidiana con uno script Python, aggiungi logging per capire il suo comportamento, scrivi qualche test unitario per una funzione complessa che hai, prova a profilare un pezzo di codice per individuare migliorie. Sperimenta con AsyncIO magari creando un semplice scraper web asincrono. Infine, prova a pacchettizzare un tuo progetto (anche piccolo) e installarlo in un ambiente virtuale, magari persino caricarlo su TestPyPI.

Python offre un ecosistema ricchissimo: man mano che ti sentirai a tuo agio con questi strumenti di base, potrai esplorare librerie e framework più specifici (web development, data science, automazione di sistema avanzata, ecc.). La cosa bella è che i principi che hai imparato – automatizzare, testare, loggare, ottimizzare – saranno validi ovunque nel tuo percorso da programmatore. Buon coding!

```
# Fine della lezione
print("Complimenti, hai completato la lezione introduttiva!")
```

1 logging — Logging facility for Python — Python 3.14.0 documentation

<https://docs.python.org/3/library/logging.html>

2 unittest — Unit testing framework — Python 3.14.0 documentation

<https://docs.python.org/3/library/unittest.html>

3 asyncio — Asynchronous I/O — Python 3.14.0 documentation

<https://docs.python.org/3/library/asyncio.html>

4 Packaging Python Projects

<https://packaging.python.org/tutorials/packaging-projects/>